Chapter 1. The Basics

PostgreSQL is an extremely powerful piece of software that introduces features you may not have seen before. Some of the features are also present in other well-known database engines, but under different names. This chapter lays out the main concepts you should know when starting to attack PostgreSQL documentation, and mentions some related terms in other databases.

We begin by pointing you to resources for downloading and installing PostgreSQL. Next, we provide an overview of indispensable administration tools followed by a review of PostgreSQL nomenclature. PostgreSQL 10 was recently released. We'll highlight some of the new features therein. We close with resources to turn to when you need additional guidance and to submit bug reports.

Why PostgreSQL?

PostgreSQL is an enterprise-class relational database management system, on par with the very best proprietary database systems: Oracle, Microsoft SQL Server, and IBM DB2, just to name a few. PostgreSQL is special because it's not just a database: it's also an application platform, and an impressive one at that.

PostgreSQL is fast. In benchmarks, PostgreSQL either exceeds or matches the performance of many other databases, both open source and proprietary.

PostgreSQL invites you to write stored procedures and functions in numerous programming languages. In addition to the prepackaged languages of C, SQL, and PL/pgSQL, you can easily enable support for additional languages such as PL/Perl, PL/Python, PL/V8 (aka PL/JavaScript), PL/Ruby, and PL/R. This support for a wide variety of languages allows you to choose the language with constructs that can best solve the problem at hand. For

instance, use R for statistics and graphing, Python for calling web services, the Python SciPy library for scientific computing, and PL/V8 for validating data, processing strings, and wrangling with JSON data. Easier yet, find a freely available function that you need, find out the language that it's written in, enable that specific language in PostgreSQL, and copy the code. No one will think less of you.

Most database products limit you to a predefined set of data types: integers, texts, Booleans, etc. Not only does PostgreSQL come with a larger built-in set than most, but you can define additional data types to suit your needs. Need complex numbers? Create a composite type made up of two floats. Have a triangle fetish? Create a coordinate type, then create a triangle type made up of three coordinate pairs. A dozenal activist? Create your own duodecimal type. Innovative types are useful insofar as the operators and functions that support them. So once you've created your special number types, don't forget to define basic arithmetic operations for them. Yes, PostgreSQL will let you customize the meaning of the symbols (+,-,/,*). Whenever you create a type, PostgreSQL automatically creates a companion array type for you. If you created a complex number type, arrays of complex numbers are available to you without additional work.

PostgreSQL also automatically creates types from any tables you define. For instance, create a table of dogs with columns such as breed, cuteness, and barkiness. Behind the scenes, PostgreSQL maintains a dogs data type for you. This amazingly useful bridge between the relational world and the object world means that you can treat data elements in a way that's convenient for the task at hand. You can create functions that work on one object at a time or functions that work on sets of objects at a time. Many third-party extensions for PostgreSQL leverage custom types to achieve performance gains, provide domain-specific constructs for shorter and more maintainable code, and accomplish feats you can only fantasize about with other database products.

Our principal advice is this: don't treat databases as dumb storage. A database such as PostgreSQL can be a full-fledged application platform. With a robust database, everything else is eye candy. Once you're versant in SQL, you'll be able to accomplish in seconds what would take a casual

programmer hours, both in coding and running time.

In recent years, we've witnessed an upsurge of NoSQL movements (though much of it could be hype). Although PostgreSQL is fundamentally relational, you'll find plenty of facilities to handle nonrelational data. The ltree extension to PostgreSQL has been around since time immemorial and provides support for graphs. The hstore extensions let you store key-value pairs. JSON and JSONB types allow storage of documents similar to MongoDb. In many ways, PostgreSQL accommodated NoSQL before the term was even coined!

PostgreSQL just celebrated its 20th birthday, dating from its christening to PostgreSQL from Postgres95. The beginnings of the PostgreSQL code-base began well before that in 1986. PostgreSQL is supported on all major operating systems: Linux, Unix, Windows, and Mac. Every year brings a new major release, offering enhanced performance along with features that push the envelope of what's possible in a database offering.

Finally, PostgreSQL is open source with a generous licensing policy. PostgreSQL is supported by a community of developers and users where profit maximization is not the ultimate pursuit. If you want features, you're free to contribute, or at least vocalize. If you want to customize and experiment, no one is going to sue you. You, the mighty user, make PostgreSQL what it is.

In the end, you will wonder why you ever used any other database, because PostgreSQL does everything you could hope for and does it for free. No more reading the licensing cost fineprint of those other databases to figure out how many dollars you need to spend if you have eight cores on your virtualized servers with *X* number of concurrent connections. No more fretting about how much more the next upgrade will cost you.

Why Not PostgreSQL?

Given all the proselytizing thus far, it's only fair that we point out situations when PostgreSQL might not be suitable.

The typical installation size of PostgreSQL without any extensions is more than 100 MB. This rules out PostgreSQL for a database on a small device or as a simple cache store. Many lightweight databases abound that could better serve your needs without the larger footprint.

Given its enterprise stature, PostgreSQL doesn't take security lightly. If you're developing lightweight applications where you're managing security at the application level, PostgreSQL security with its sophisticated role and permission management could be overkill. You might consider a single-user database such as SQLite or a database such as Firebird that can be run either as a client server or in single-user embedded mode.

All that said, it is a common practice to combine PostgreSQL with other database types. One common combination you will find is using Redis or Memcache to cache PostgreSQL query results. As another example, SQLite can be used to store a disconnected set of data for offline querying when PostgreSQL is the main database backend for an application.

Finally, many hosting companies don't offer PostgreSQL on a shared hosting environment, or they offer an outdated version. Most still gravitate toward the impotent MySQL. To a web designer, for whom the database is an afterthought, MySQL might suffice. But as soon as you learn to write any SQL beyond a single-table select and simple joins, you'll begin to sense the shortcomings of MySQL. Since the first edition of this book, virtualization has resown the landscape of commerical hosting, so having your own dedicated server is no longer a luxury, but the norm. And when you have your own server, you're free to choose what you wish to have installed. PostgreSQL bodes well with the popularity of cloud computing such as Platform as a Service (PaaS) and Database as a Service (DbaaS). Most of the major PaaS and DbaaS providers offer PostgreSQL, notably Heroku, Engine Yard, Red Hat OpenShift, Amazon RDS for PostgreSQL, Google Cloud SQL for PostgreSQL, Amazon Aurora for PostgreSQL, and Microsoft Azure for PostgreSQL.

Where to Get PostgreSQL

Years ago, if you wanted PostgreSQL, you had to compile it from source. Thankfully, those days are long gone. Granted, you can still compile from source, but using packaged installers won't make you any less cool. A few clicks or keystrokes, and you're on your way.

If you're installing PostgreSQL for the first time and have no existing database to upgrade, you should install the latest stable release version for your OS. The downloads page for the PostgreSQL core distribution maintains a listing of places where you can download PostgreSQL binaries for various OSes. In Appendix A, you'll find useful installation instructions and links to additional custom distributions.

Administration Tools

Four tools widely used with PostgreSQL are psql, pgAdmin, phpPgAdmin, and Adminer. PostgreSQL core developers actively maintain the first three; therefore, they tend to stay in sync with PostgreSQL releases. Adminer, while not specific to PostgreSQL, is useful if you also need to manage other relational databases: SQLite, MySQL, SQL Server, or Oracle. Beyond the four that we mentioned, you can find plenty of other excellent administration tools, both open source and proprietary.

psql

psql is a command-line interface for running queries and is included in all distributions of PostgreSQL (see "psql Interactive Commands"). psql has some unusual features, such as an import and export command for delimited files (CSV or tab), and a minimalistic report writer that can generate HTML output. psql has been around since the introduction of PostgreSQL and is the tool of choice for many expert users, for people working in consoles without a GUI, or for running common tasks in shell scripts. Newer converts favor GUI tools and wonder why the older generation still clings to the command line.

pgAdmin

pgAdmin is a popular, free GUI tool for PostgreSQL. Download it separately from PostgreSQL if it isn't already packaged with your installer. pgAdmin runs on all OSes supported by PostgreSQL.

Even if your database lives on a console-only Linux server, go ahead and install pgAdmin on your workstation, and you'll find yourself armed with a fantastic GUI tool.

pgAdmin recently entered its fourth release, dubbed pgAdmin4. pgAdmin4 is a complete rewrite of pgAdmin3 that sports a desktop as well as a web server application version utilizing Python. pgAdmin4 is currently at version 1.5. It made its debut at the same time as PostgreSQL 9.6 and is available as part of several PostgreSQL distributions. You can run pgAdmin4 as a desktop application or via a browser interface.

An example of pgAdmin4 appears in Figure 1-1.

If you're unfamiliar with PostgreSQL, you should definitely start with pgAdmin. You'll get a bird's-eye view and appreciate the richness of PostgreSQL just by exploring everything you see in the main interface. If you're deserting Microsoft SQL Server and are accustomed to Management Studio, you'll feel right at home.

pgAdmin4 still has a couple of pain points compared to pgAdmin3, but its feature set is ramping up quickly and in some ways already surpasses pgAdmin3. That said, if you are a long-time user of pgAdmin3, you might want to go for the pgAdmin3 Long Time support (LTS) version supported and distributed by BigSQL, and spend a little time test-driving pgAdmin4 before you fully commit to it. But keep in mind that the pgAdmin project is fully committed to pgAdmin4 and no longer will make changes to pgAdmin3.

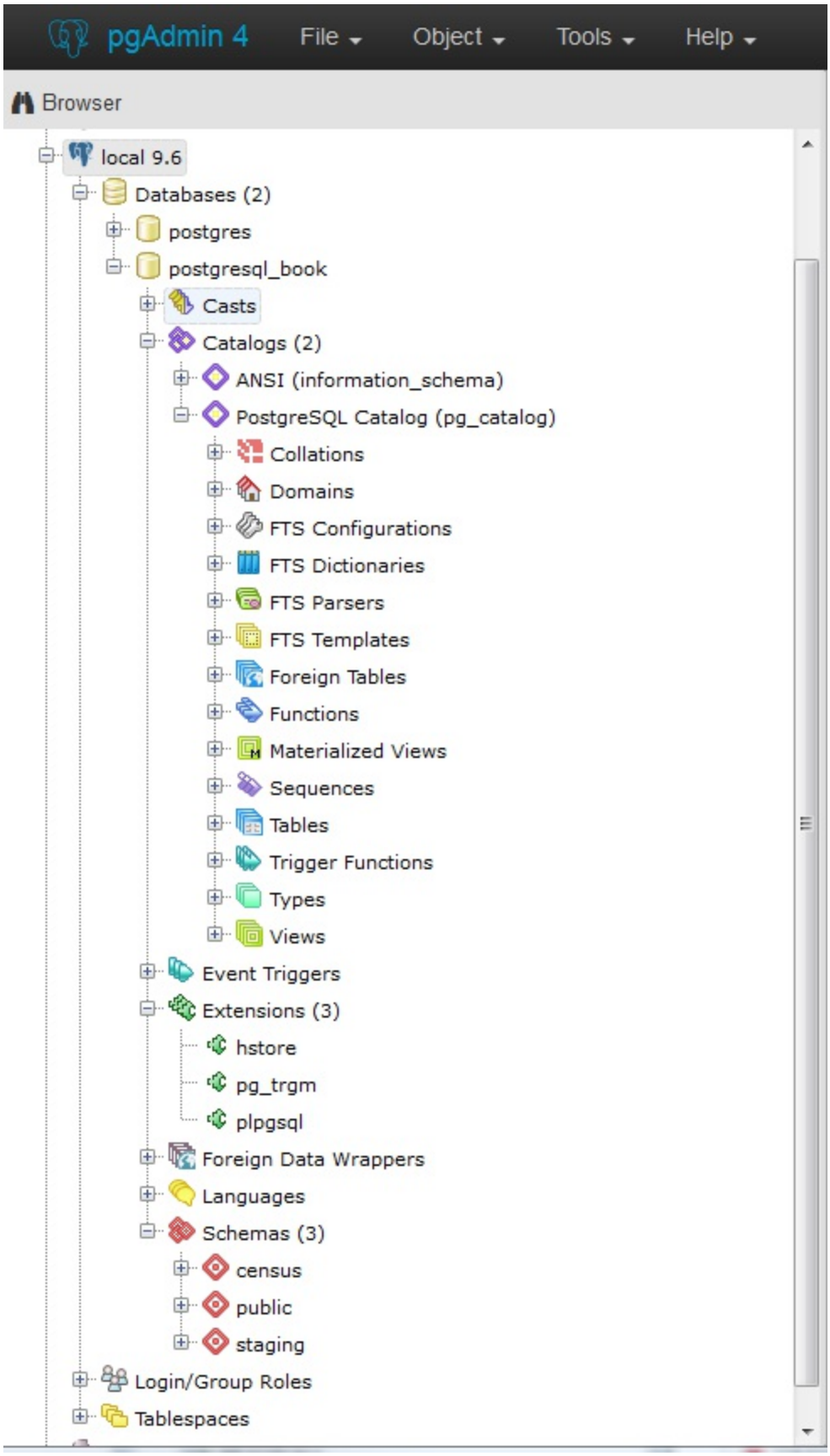
pgAdmin 4
File
Object
Tools
Help
Browser
local 9.6
Databases (2)
postgres
postgresql_book
Casts
Catalogs (2)
ANSI (information_schema)
PostgreSQL Catalog (pg_catalog)
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Sequences
Tables
Trigger Functions
Types
Views
Event Triggers
Extensions (3)
hstore
pg_trgm
plpgsql
Foreign Data Wrappers
Languages
Schemas (3)
census
public
staging
Login/Group Roles
Tablespaces

Figure 1-1. pgAdmin4 tree browser

phpPgAdmin

phpPgAdmin, pictured in Figure 1-2, is a free, web-based administration tool patterned after the popular phpMyAdmin. phpPgAdmin differs from phpMyAdmin by including ways to manage PostgreSQL objects such as schemas, procedural languages, casts, operators, and so on. If you've used phpMyAdmin, you'll find phpPgAdmin to have the same look and feel.

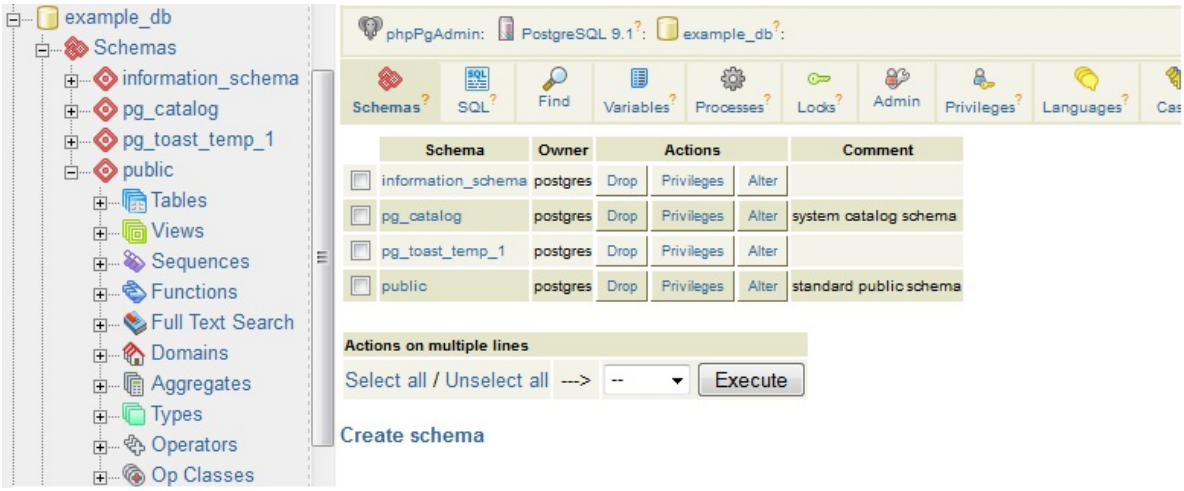

Figure 1-2. phpPgAdmin

Adminer

If you manage other databases besides PostgreSQL and are looking for a unified tool, Adminer might fit the bill. Adminer is a lightweight, open source PHP application with options for PostgreSQL, MySQL, SQLite, SQL Server, and Oracle, all delivered through a single interface.

One unique feature of Adminer we're impressed with is the relational diagrammer that can produce a schematic layout of your database schema, along with a linear representation of foreign key relationships. Another hassle-reducing feature is that you can deploy Adminer as a single PHP file.

Figure 1-3 is a screenshot of the login screen and a snippet from the diagrammer output. Many users stumble in the login screen of Adminer

because it doesn't include a separate text box for indicating the port number. If PostgreSQL is listening on the standard 5432 port, you need not worry. But if you use some other port, append the port number to the server name with a colon, as shown in Figure 1-3.

Adminer is sufficient for straightforward querying and editing, but because it's tailored to the lowest common denominator among database products, you won't find management applets that are specific to PostgreSQL for such tasks as creating new users, granting rights, or displaying permissions. Adminer also treats each schema as a separate database, which severely reduces the usefulness of the relational diagrammer if your relationships cross schema boundaries. If you're a DBA, stick to pgAdmin or psql.

Figure 1-3. Adminer

PostgreSQL Database Objects

So you installed PostgreSQL, fired up pgAdmin, and expanded its browse tree. Before you is a bewildering display of database objects, some familiar and some completely foreign. PostgreSQL has more database objects than most other relational database products (and that's before add-ons). You'll probably never touch many of these objects, but if you dream up something new, more likely than not it's already implemented using one of those esoteric objects. This book is not even going to attempt to describe all that you'll find in a standard PostgreSQL install. With PostgreSQL churning out

features at breakneck speed, we can't imagine any book that could possibly do this. We limit our quick overview to those objects that you should be familiar with:

Databases

Each PostgreSQL service houses many individual databases.

Schemas

Schemas are part of the ANSI SQL standard. They are the immediate next level of organization within each database. If you think of the database as a country, schemas would be the individual states (or provinces, prefectures, or departments, depending on the country). Most database objects first belong to a schema, which belongs to a database. When you create a new database, PostgreSQL automatically creates a schema named *public* to store objects that you create. If you have few tables, using public would be fine. But if you have thousands of tables, you should organize them into different schemas.

Tables

Tables are the workhorses of any database. In PostgreSQL, tables are first citizens of their respective schemas, which in turn are citizens of the database.

PostgreSQL tables have two remarkable talents: first, they are inheritable. Table inheritance streamlines your database design and can save you endless lines of looping code when querying tables with nearly identical structures. Second, whenever you create a table, PostgreSQL automatically creates an accompanying custom data type.

Views

Almost all relational database products offer views as a level of abstraction from tables. In a view, you can query multiple tables and present additional derived columns based on complex calculations. Views are generally read-only, but PostgreSQL allows you to update the underlying data by updating the view, provided that the view draws from